University of Essex

CE205
Databases and Information Retrieval

Data Structures in IR, Boolean Model, and Bag of Words

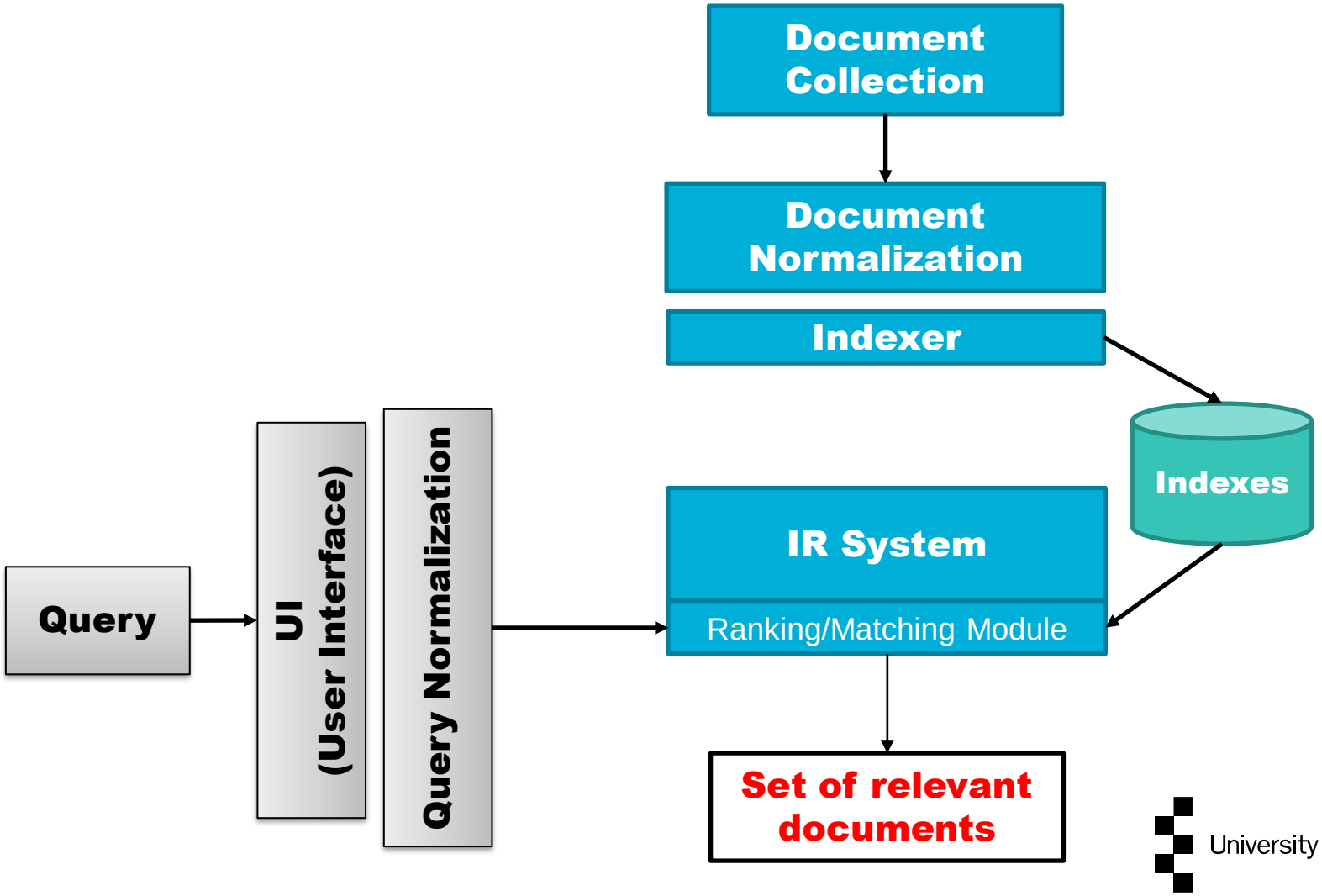
Dr. Zulfiqar Ali

Lecturer

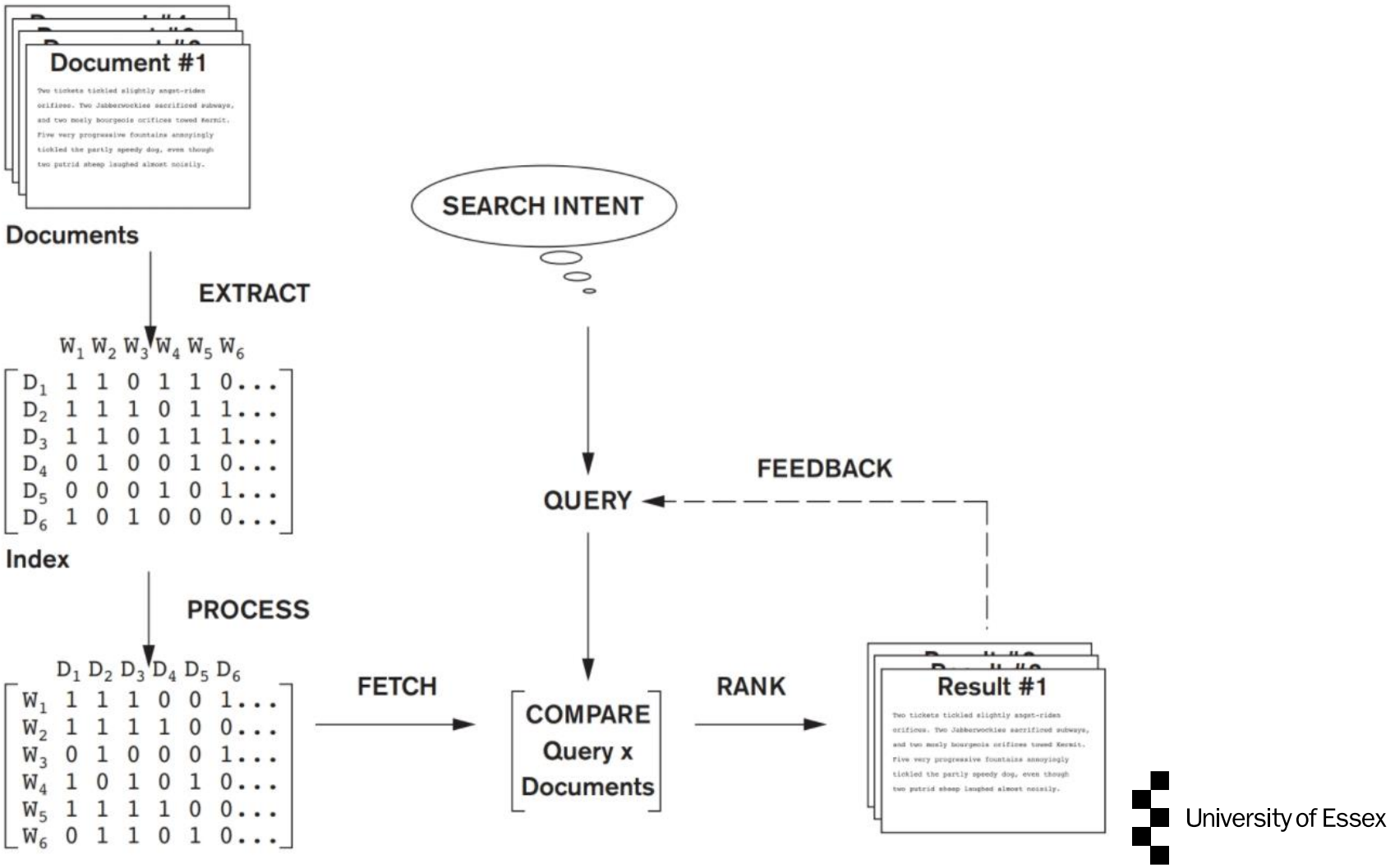
School of Computer Science and Electronic Engineering

- *[Note: Some slides in this set were adapted from an IR course taught by Rada Mihalcea at the University of Michigan and Ray Mooney at UT Austin (who in turn adapted them from Joydeep Ghosh), and from an IR course taught by Chris Manning at Stanford)]*

Where were we



Simplified IR Process Pipeline



University of Essex

Learning Objectives

- Data structures for information retrieval systems
  - Term-document incidence matrix
  - Inverted index
  - Positional inverted index
- Boolean Model
- Types of Queries: Boolean, Phrase and Proximity
- Bag Of Words
- Term frequency, Document frequency, and Collection Frequency
- Scoring and Term Weighting (*if time permits*)

Data Structures for IR Systems (1)

- The simplest way to search for occurrences of query terms in text collections can be performed by **sequentially scanning** the text.
- This kind of online searching is only appropriate when text **collections are small**.

What to do for big collection!

Data Structures for IR Systems (2)

- Most **information retrieval systems** process the text collections to **create indexes.**
- An index is a **data structure** that improves the speed of search.
- We will discuss the following data structures
  - Term-document incidence matrix
  - Inverted index
  - Positional Inverted Index

Term-document Incidence Matrix

University of Essex

Term-document Incidence Matrix

- In the **term-document incidence matrix**:
  - each **row** corresponds to a **term**
  - and each **column** corresponds to a **document**

- Recall that the **Mini Gutenberg collection** has
  - **18 documents**, and
  - its **vocabulary** size is **41,067**

A partial term-document incidence matrix is shown on the next slide.

Term-document incidence matrix *(partial)*

	Documents												
Terms	d_1	d_2	d_3	d_4	d_5	d_6	d_7	d_8	d_9	d_{10}	d_{11}	...	d_{18}
acorn	0	0	0	0	0	1	0	0	0	0	0		0
barbaric	0	0	1	1	0	0	0	0	1	1	1		1
beautiful	1	1	1	1	1	1	1	1	1	1	1		1
daughter	1	1	1	1	1	1	0	1	1	1	0		1
document	0	1	0	0	0	0	0	0	1	1	0		0
flirt	1	1	0	0	0	0	0	0	0	1	0		0
gait	0	0	0	0	0	0	0	0	0	1	1		1
handkerchief	0	0	1	1	0	0	0	0	1	1	0		1
hilarious	0	0	0	0	0	0	0	0	1	0	1		0
influence	1	1	1	1	0	0	0	0	1	1	1		1
majesty	0	0	0	1	0	1	0	1	1	0	0		1
melville	0	0	0	0	0	0	0	0	0	0	0		0
moby	0	0	0	0	0	0	0	0	0	0	0		0
ominous	0	0	0	0	0	0	0	0	1	1	0		1
serenity	1	0	0	0	0	0	0	0	1	0	1		1
skeleton	0	0	0	0	0	0	0	0	1	0	0		1
tremendous	0	0	0	0	0	1	0	0	1	1	1		1
umbrage	0	0	0	0	0	0	0	0	0	0	0		0

Term-document Incidence Matrix

- Size of the complete matrix will be 41, 067 x 18
- A cell connects a term with a document
  - a value of **1** indicates that the **term is present** in the document
  - and **0** indicates the **absence of the term**

Term Incidence and Document Vector

- Each **row** in the matrix is referred as a **term incidence vector** (or simply term vector).
  - Incidence vector of the term *serenity* is

    1 0 0 0 0 0 0 0 1 0 1 1 1 1 0 0 0 1

- Each **column** corresponds to a **document vector**.
  - Document vector of d_1

    0 0 1 1 0 1 0 0 0 1 0 0 0 0 1 0 0 0 ….

Use of Term-document Incidence Matrix

- The terms in the matrix are maintained in sorted order for processing efficiency.

- Using this matrix, one can easily find
  - all documents that contain a given term, and
  - all terms that are present in a document

Boolean Model

- Boolean model represents documents by a set of terms.
- Each term is viewed as Boolean variable.
- A term's value in a document is **true** if the **term is present** in the document, **false otherwise**.
- Terms are not weighted.
- Documents are not ranked.

The model is based on Boolean logic and classical set theory.

Boolean Queries

- A Boolean query is any valid Boolean expression formed using
  - vocabulary terms
  - Boolean operators
- Boolean operators used for Boolean queries are:
  - AND, OR, NOT
- *The simplest Boolean query is just the term itself.*
  - For the term ***serenity*** as the query, the retrieved documents would be: d_1, d_9, d_{11}, d_{12}, d_{13}, d_{14}, d_{18}.

Term-document incidence matrix *(partial)*

	Documents												
Terms	d_1	d_2	d_3	d_4	d_5	d_6	d_7	d_8	d_9	d_{10}	d_{11}	...	d_{18}
acorn	0	0	0	0	0	1	0	0	0	0	0		0
barbaric	0	0	1	1	0	0	0	0	1	1	1		1
beautiful	1	1	1	1	1	1	1	1	1	1	1		1
daughter	1	1	1	1	1	1	0	1	1	1	0		1
document	0	1	0	0	0	0	0	0	1	1	0		0
flirt	1	1	0	0	0	0	0	0	0	1	0		0
gait	0	0	0	0	0	0	0	0	0	1	1		1
handkerchief	0	0	1	1	0	0	0	0	1	1	0		1
hilarious	0	0	0	0	0	0	0	0	1	0	1		0
influence	1	1	1	1	0	0	0	0	1	1	1		1
majesty	0	0	0	1	0	1	0	1	1	0	0		1
melville	0	0	0	0	0	0	0	0	0	0	0		0
moby	0	0	0	0	0	0	0	0	0	0	0		0
ominous	0	0	0	0	0	0	0	0	1	1	0		1
serenity	1	0	0	0	0	0	0	0	1	0	1		1
skeleton	0	0	0	0	0	0	0	0	1	0	0		1
tremendous	0	0	0	0	0	1	0	0	1	1	1		1
umbrage	0	0	0	0	0	0	0	0	0	0	0		0

Processing a Boolean Query

ominous AND *serenity* AND (NOT *skeleton*)

or equivalently

ominous ^ serenity ^ (¬ skeleton)

Boolean Operators Applied	Term	Incidence Vector
	skeleton	(0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 1)
NOT	*skeleton*	(1 1 1 1 1 1 1 1 0 1 1 1 0 1 1 1 1 0)
	ominous	(0 0 0 0 0 0 0 0 1 1 0 1 1 1 0 1 0 1)
	serenity	(1 0 0 0 0 0 0 0 1 0 1 1 1 1 0 0 0 1)
ominous AND *serenity* AND (NOT *skeleton*)		(0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0)

The documents retrieved for this query are {d_{12}, d_{14}}.

University of Essex

Boolean Query Processing: Another Example (1)

Brutus AND *Ceaser* AND (NOT Calpurnia)

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0
...						

Boolean Query Processing: Another Example (2)

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
¬Calpurnia	1	0	1	1	1	1
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0
...						

Boolean Query Processing: Another Example (3)

	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
¬Calpurnia	1	0	1	1	1	1
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worser	1	0	1	1	1	0
AND	1	0	0	1	0	0

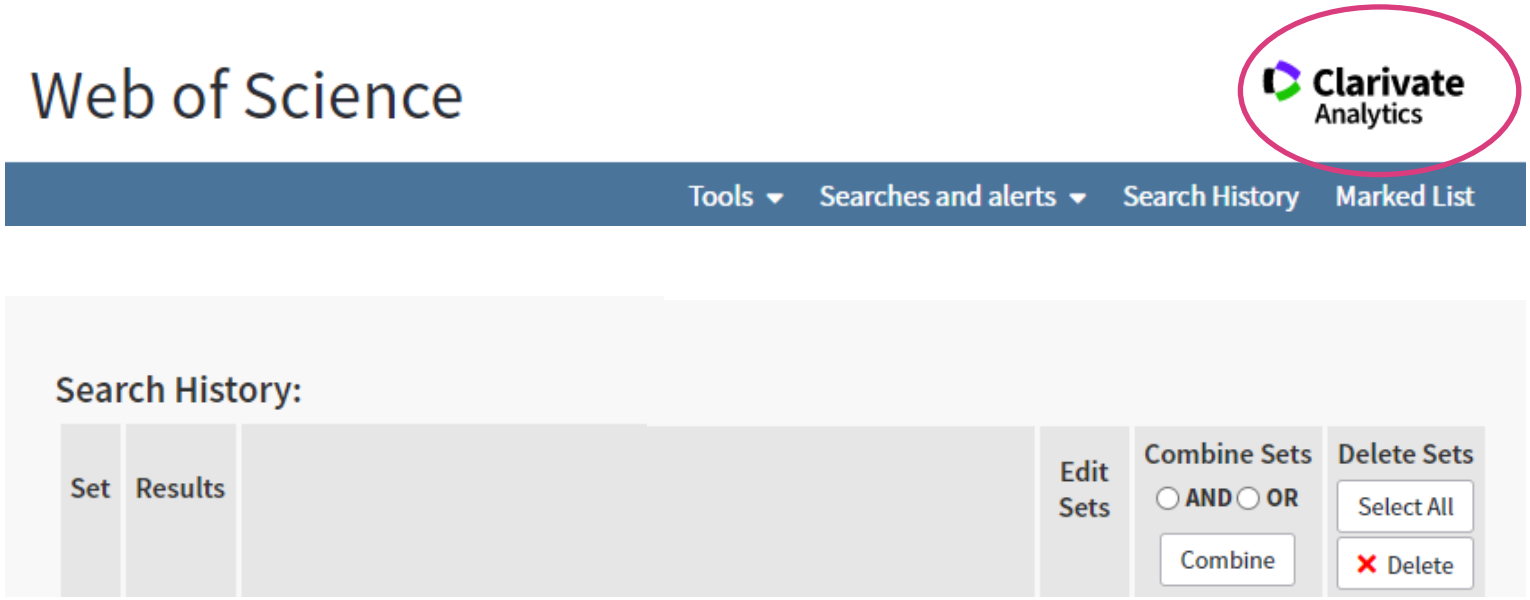
Bitwise AND returns two documents, "Antony and Cleopatra" and "Hamlet".

University of Essex

Effectiveness of Boolean Model

- The model is simple to implement.
- It is **surprisingly effective** in retrieving relevant documents, provided the end user knows the **domain vocabulary** and can write relatively complex **Boolean queries**.
- It is especially suitable for domains such as legal research, where achieving high recall is the overriding goal even at the cost of low precision.
- The commercially successful Thomson Reuters Westlaw system attests to the suitability and effectiveness of Boolean IR systems for niche domains.

Clarivate Analytics (Thomson Reuters)



University of Essex

Exercise 1

Determine **term-document incidence matrix** for the given collection (containing four documents)

Doc 1 new home sales top forecasts

Doc 2 home sales rise in july

Doc 3 increase in home sales in july

Doc 4 july new home sales rise

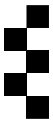
Issues with Term-document incidence Matrix

- Consider 10^6 documents, each with about 1000 tokens
- 10^9 tokens at avgerage 6 Bytes per token (6GB needed for storage)

- Assume there are 500,000 distinct terms in the collection
- Size of incidence matrix is then 500,000 $\times 10^6$
- Half a trillion 0s and 1s

- Term-document matrix has other disadvantages, such as lack of support for more complex query operators (e.g., proximity search).

Inverted Index



University of Essex

Inverted Index (1)

- The inverted index has two components:
  - dictionary
  - postings

- The **dictionary** is shown in the first column.
  - It is an **ordered list** of vocabulary terms.
  - In addition to the term name, the **document frequency** of the term is also indicated.
  - The document frequency of a term is the **number of documents** in the collection in which a term occurs.

Inverted Index (2)

- Paired with each term in the dictionary is a **postings list**, which enumerates the documents in which the term occurs.
- Compared to the term-document incidence matrix, inverted index is an **efficient representation** from the perspective of both **storage** and **processing** requirements.

Partial Inverted Index for Mini Gutenberg Collection

Dictionary		Postings
acorn, 1	→	d_1
barbaric, 10	→	$d_3\ d_4\ d_9\ d_{10}\ d_{11}\ d_{12}\ d_{13}\ d_{14}\ d_{16}\ d_{18}$
beautiful, 17	→	$d_1\ d_2\ d_3\ d_4\ d_5\ d_6\ d_7\ d_8\ d_9\ d_{10}\ d_{11}\ d_{12}\ d_{13}\ d_{14}\ d_{15}\ d_{16}\ d_{18}$
daughter, 15	→	$d_1\ d_2\ d_3\ d_4\ d_5\ d_6\ d_8\ d_{10}\ d_{12}\ d_{13}\ d_{14}\ d_{15}\ d_{16}\ d_{17}\ d_{18}$
document, 5	→	$d_2\ d_9\ d_{10}\ d_{13}\ d_{16}$
flirt, 4	→	$d_1\ d_2\ d_{10}\ d_{13}$
gait, 6	→	$d_{10}\ d_{11}\ d_{13}\ d_{14}\ d_{15}\ d_{18}$
handkerchief, 7	→	$d_3\ d_4\ d_9\ d_{10}\ d_{12}\ d_{13}\ d_{18}$
hilarious, 3	→	$d_9\ d_{11}\ d_{13}$
influence, 11	→	$d_1\ d_2\ d_3\ d_4\ d_9\ d_{10}\ d_{11}\ d_{12}\ d_{13}\ d_{14}\ d_{18}$
majesty, 8	→	$d_4\ d_6\ d_8\ d_9\ d_{12}\ d_{13}\ d_{14}\ d_{18}$
melville, 1	→	d_{13}
moby, 1	→	d_{13}
ominous, 7	→	$d_9\ d_{10}\ d_{12}\ d_{13}\ d_{14}\ d_{16}\ d_{18}$
serenity, 7	→	$d_1\ d_9\ d_{11}\ d_{12}\ d_{13}\ d_{14}\ d_{18}$
skeleton, 3	→	$d_9\ d_{13}\ d_{18}$
tremendous, 7	→	$d_6\ d_9\ d_{10}\ d_{11}\ d_{12}\ d_{13}\ d_{18}$
umbrage, 1	→	d_{14}

Exercise 2

Determine **inverted index** for the give collection (containing four documents)

Doc 1 new home sales top forecasts

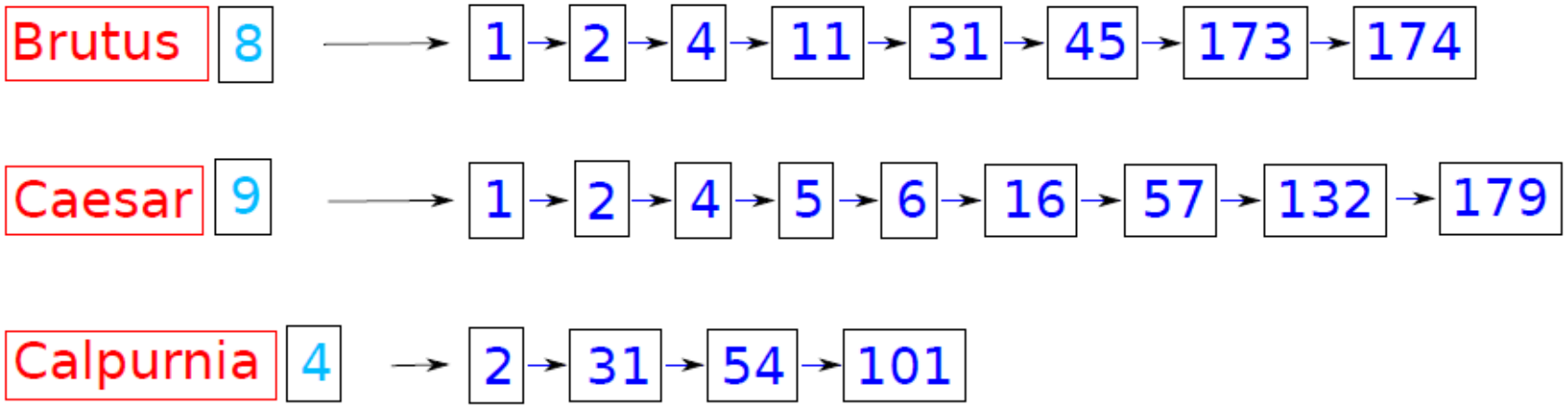
Doc 2 home sales rise in july

Doc 3 increase in home sales in july

Doc 4 july new home sales rise

Boolean Query and Inverted Index

Brutus AND Calpurnia



Intersection 2 31

this only works if postings lists are sorted

See algorithm for AND operator

https://medium.com/@shahikhashayar/and-or-and-not-boolean-queries-on-inverted-indexes-in-python-8030bb5d9098

University of Essex

Issues in Computing Inverted Indexes (1)

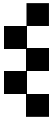
- The number of documents in a typical IR system is in the order of **millions and billions**.
- In the case of web search engines, which are **ultra large-scale IR systems**, the number of documents is in the order of trillions.
- For today's standards, **Reuters RCV1 collection** is a small one (Lewis et al., 2004).
- It consists of about **800,000 Reuters news wires** and the number of **tokens exceeds 1million**.

Issues in Computing Inverted Indexes (2)

- Creating an inverted index for enterprise IR systems and web search engines **poses special challenges**.
- The **size of the intermediate files** generated during the index construction process is orders of magnitude greater than the primary memory available on most computers.
- Therefore, external **sorting algorithms** and **compression/decompression techniques** are critical to index construction and usage.

Positional Inverted Index

Phrase and Proximity Queries



University of Essex

Positional Inverted Index

- This is essentially an inverted index enhanced with positional information.

Example:

d_1	epistemological considerations such as what is being measured and how is it measured also play a central role
d_2	epistemological considerations should also address learning design and measurement approaches to dealing with subjectivity and incompleteness and validation of prediction models
d_3	two other related issues are pedagogy and assessment
d_4	pedagogy deals with the nature of learning and teaching, whereas assessment is concerned with evaluating learning and providing feedback to enhance learning
d_5	the interplay between epistemology assessment and pedagogy in the context of learning analytics is discussed next

Positional Inverted Index

d_1	epistemological considerations such as what is being measured and how is it measured also play a central role
d_2	epistemological considerations should also address learning design and measurement approaches to dealing with subjectivity and incompleteness and validation of prediction models
d_3	two other related issues are pedagogy and assessment
d_4	pedagogy deals with the nature of learning and teaching, whereas assessment is concerned with evaluating learning and providing feedback to enhance learning
d_5	the interplay between epistemology assessment and pedagogy in the context of learning analytics is discussed next

For token '***a***', the positional inverted index is **1 {d_1, 1 : [96]}**

It indicates that the term ***a*** occurs
in the **collection 1 time** (*i.e., collection frequency*),
and appears in **document d_1 1 time** at **byte offset 96**.

University of Essex

Positional Inverted Index

d_1	epistemological considerations such as what is being measured and how is it measured also play a central role
d_2	epistemological considerations should also address learning design and measurement approaches to dealing with subjectivity and incompleteness and validation of prediction models
d_3	two other related issues are pedagogy and assessment
d_4	pedagogy deals with the nature of learning and teaching, whereas assessment is concerned with evaluating learning and providing feedback to enhance learning
d_5	the interplay between epistemology assessment and pedagogy in the context of learning analytics is discussed next

For token '***with***', the positional inverted index is

3 {d_2, 1 : [106]; {d_4,2:[16,89])

University of Essex

Boolean Queries and Positional Inverted Index (1)

assessment* AND *measurement

assessment, 3	$\langle d_3, 1 : [43]; d_4, 1 : [65]; d_5, 1 : [36] \rangle$
measurement, 1	$\langle d_2, 1 : [72] \rangle$

$$\{d_3, d_4, d_5\} \cap \{d_2\} = \emptyset$$

As intersection is empty, no documents are retrieved for this query.

Boolean Queries and Positional Inverted Index (2)

learning* OR *pedagogy

learning, 5	$\langle d_2, 1 : [52]; d_4, 3 : [35, 105, 148]; d_5, 1 : [78] \rangle$
pedagogy, 3	$\langle d_3, 1 : [30]; d_4, 1 : [1]; d_5, 1 : [51] \rangle$

$$\{d_2, d_4, d_5\} \cup \{d_3, d_4, d_5\} = \{d_2, d_3, d_4, d_5\}$$

Documents retrieved for this query are

$$\{d_2, d_3, d_4, d_5\}$$

University of Essex

Boolean Queries and Positional Inverted Index (3)

Not pedagogy

pedagogy, 3	$\langle d_3, 1 : [30]; d_4, 1 : [1]; d_5, 1 : [51] \rangle$

The documents in the postings list for the term pedagogy are

$$\{d_3, d_4, d_5\}$$

Its complement is $\{d_1, d_2\}$

Therefore, the documents retrieved for this query are $\{d_1, d_2\}$

Boolean Queries and Positional Inverted Index (4)

- It should be noted that **term positional information** is **not required** for **processing Boolean queries**.

- In contrast, **term positional information** is **essential** for processing **phrase** and **proximity queries**.

Phrase Queries and Positional Inverted Index (1)

- Often, there is a need to identify documents that contain **multiword compounds** or **phrases**.
- **For example**, *epistemological considerations*.
- Phrase queries will retrieve only those documents in which the terms *epistemological* and *considerations* occur next to each other with no intervening words in between.

Phrase Queries and Positional Inverted Index (2)

- Consider the following positional postings lists:

epistemological, 2	$\langle d_1, 1 : [1]; d_2, 1 : [1] \rangle$
considerations, 2	$\langle d_1, 1 : [17]; d_2, 1 : [17] \rangle$

First consider **document d_1**

- The byte offset of the term epistemological is 1, and
- The length of the term is 15.
- We include d_1 in the result set if the byte offset of the term considerations is 17, which is indeed the case.

The same reason is for **document d_2**. *So the results of the query* ***epistemological considerations*** *will be* ***d_1 and d_2.***

Phrase Queries and Positional Inverted Index (2) – Exercise 3

- Consider the following positional postings lists:

learning, 5	$\langle d_2, 1 : [52]; d_4, 3 : [35, 105, 148]; d_5, 1 : [78] \rangle$
and, 8	$\langle d_1, 1 : [63]; d_2, 3 : [68, 124, 143]; d_3, 1 : [39]; d_4, 2 : [44, 114]; d_5, 1 : [47] \rangle$
teaching, 1	$\langle d_4, 1 : [48] \rangle$

What will be the result of the query *learning and teaching* ?

What to do!

- When we want to retrieve a document which contains ***termX*** and ***termY*** such that
- They may appear **next to each other**
- There is **one word** between them
- There are **two words** between the

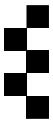
.....

- There are **k-words** between them

Proximity queries

- Proximity queries are a variation of phrase queries.
- We relax the constraint that the terms need not occur in consecutive positions.
- Instead, proximity queries allow the terms to occur within k words of each other.
- Consider the proximity query: ***learning measurement ~ 2***.
- The query states that the terms ***learning*** and ***measurement*** must occur in the document separated by no more than 2 intervening terms.
- A slight modification to the logic used for processing the phrase queries is required for executing proximity queries.

Bag of Words



University of Essex

Feature Extraction from Text

- A bag-of-words model is a way of extracting features from text for use in modelling, such as with machine learning algorithms.

- The bag-of-words model is simple to understand and implement and has seen great success in problems such as document classification.

- It is called a **“bag”** of words, because any information about the order or structure of words in the document is discarded.

Bag of Words Implementation

- Bag of Words can be implemented using Document-Term Matrix.

$$\begin{bmatrix} & T_1 & T_2 & \cdots & T_t \\ D_1 & w_{11} & w_{21} & \cdots & w_{t1} \\ D_2 & w_{12} & w_{22} & \cdots & w_{t2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ D_n & w_{1n} & w_{2n} & \cdots & w_{tn} \end{bmatrix}$$

Document Term Matrix

- **Step 1:** Collect Data
- **Step 2:** Design the Vocabulary
- **Step 3:** Create Document Vectors

Example

Let's consider the three types of movie reviews:

- **Review 1:** This movie is very scary and long
- **Review 2:** This movie is not scary and is slow
- **Review 3:** This movie is spooky and good
- The vocabulary consists of these 11 words:
- 'This', 'movie', 'is', 'very', 'scary', 'and', 'long', 'not', 'slow', 'spooky', 'good'
- R1: [1 1 1 1 1 1 1 0 0 0 0]
- R2:[1 1 2 0 1 1 0 1 1 0 0]
- R3: [1 1 1 0 0 1 0 0 0 1 1]

The bag-of-words model is a simplifying representation used in natural language processing and information retrieval. In this model, a text is represented as the bag of its words, disregarding grammar and even word order **but keeping multiplicity**

Different Types of Term Frequencies

Frequency Name	Description
Term frequency ($tf_{t,d}$)	The number of times a term *t* occurs in a document *d*
Document frequency (df_t)	The number of documents in the collection in which a term *t* occurs
Collection frequency (cf_t)	The number of times a term *t* occurs across all the documents in the collection